



Senior SDET – AI-Driven API Automation & CI Gating Challenge

Tldr;

Please still read through the whole testing process below, but what we are looking is for you to do

1. Create a pipeline/automation using whatever tool you are comfortable with, that should act as a quality gate
2. The automation should incorporate the Playwright template, Swagger file and output the LLM generated test using Playwright
3. You should use this generated Playwright to test the API itself
4. Generate an end to end front end test, using the playwright template and LLM to test the end to end flow of the checkout page



Time Expectation

2–3 hours

This exercise evaluates your ability to design and implement an AI-assisted API testing system integrated into CI/CD.

We are assessing engineering judgment, automation architecture, CI integration, and quality gate design — not the number of test cases written.



What We Provide

You will receive:

- A simple Go-based API repository
- A simple frontend checkout page that allows:
 - Submitting shipping name
 - Shipping address
 - Email
 - Completing a checkout flow
- A Swagger / OpenAPI specification file
- A preconfigured Playwright API testing template
- Basic run instructions
- A temporary Gemini API key

Objective

Design an automated quality gate where:

1. API tests are generated from the Swagger file using an LLM
2. End-to-End (E2E) frontend tests are generated using an LLM
3. Test generation and execution are integrated into CI
4. Pull Requests fail if tests fail
5. The workflow can serve as a production deployment gate

Important Note:

1. **You must not modify the Playwright configuration template.**
2. **All tests must be generated programmatically using an LLM.**
3. **Manual handwritten or manually copy-pasted tests are not allowed.**

Suggested submission Layout

None

```
.
├── README.md
├── scripts/
│   └── github_action.yaml
└── generated_test/
    ├── utils/
    ├── test/
    └── .../
```

Put the generated test in the generated_test folder,

Required if you are using tool other than Gemini

If you are using gemini, it is optional but highly recommended

For the scripts folder

Put in any script/automation/pipeline script that you have created. This can be github action workflows.yaml, codepipeline buildspec.yml etc. Please also include any supporting script here if you reference a shell script, python script etc in your pipeline.

Core Requirements

1 Programmatic LLM-Based Test Generation

You must implement a generation workflow that:

- Uses the provided Gemini API key or any LLM that you are comfortable with if you have the API key
- References or parses the Swagger file
- Generates Playwright API test files automatically
- Writes those test files into the appropriate test directory

Manual handwritten or manually copy-pasted tests are **not allowed**.

Your repository must include:

- The generation script
- The prompt template(s)
- Clear instructions for regenerating tests

The generation process must be reproducible and executable.

1A — Programmatic LLM-Based E2E Test Generation

In addition to API tests, your generation workflow must:

- Analyze the frontend checkout flow
- Generate Playwright E2E test(s)
- Cover at least one complete flow:
 - Fill in name
 - Fill in shipping address
 - Fill in email
 - Submit checkout
 - Validate successful completion state

All E2E tests must:

- Be generated programmatically using an LLM
- Be written automatically to the test directory
- Use the provided Playwright template
- Not be manually written or copy-pasted

We are evaluating your ability to:

- Translate business flow into automated test logic
- Handle selectors robustly

- Validate user-visible outcomes

2 CI-Integrated Quality Gate (Required)

Your CI pipeline must:

- Execute the test generation step
- Run both:
 - Playwright API tests
 - Playwright E2E frontend tests
- Fail the Pull Request if tests fail
- Produce test logs or artifacts

You may use:

- GitHub Actions
- AWS CodeBuild
- Another CI tool of your choice

The generation step must be part of the CI workflow — not just a local script.

We are evaluating your ability to design a complete API + UI quality gate.

3 Orchestration & Reproducibility

Your solution must clearly demonstrate:

- How test generation is triggered
- How tests are executed
- How CI coordinates these steps

You may use:

- npm scripts
- Makefile
- Shell scripts
- Docker
- Any reasonable orchestration method

We are evaluating clarity and engineering maturity.

Secret Handling

A temporary Gemini API key is provided.

You must:

- Use environment variables
- Avoid hardcoding secrets
- Avoid committing secrets
- Document how the key is injected locally and in CI

Improper secret handling will be considered a failure.



README Requirements

Your README must explain:

1. Your LLM prompt design
2. How Swagger is translated into Playwright tests
3. How the generation process works
4. How CI gates production
5. How this approach scales to many endpoints
6. How you would prevent or detect incorrect / hallucinated AI-generated tests
7. How you would handle flaky or unstable tests
8. How your LLM generates E2E selectors and ensures stability
9. How you would prevent flaky frontend tests

Keep it concise but thoughtful.



Bonus (Senior-Level Signal)

Extra credit for:

- Schema-aware assertions
- Validation or guardrails on LLM output before saving tests
- Deterministic generation strategy
- Parallel execution
- Rate-limit handling
- Dockerized CI workflow
- Idempotent test regeneration
- Intelligent caching of generated tests



What We Are Looking For

Strong candidates will:

- Treat LLM generation as a pipeline component, not a manual tool
- Integrate generation and execution into CI
- Design reproducible workflows
- Anticipate AI failure modes
- Apply secure CI and secret management practices
- Think about scaling and long-term maintainability